

BÁO CÁO KẾT QUẢ THỰC HÀNH MINI PROJECT

Môn học: Java Web Service

Tên dự án: Session13 Employee Management API – Hệ thống quản lý nhân viên

Thời gian thực hành: 120p

Họ và tên sinh viên: Phạm Tiến Hưng

Mã sinh viên: KS-242

Lớp: HN-K24-CNTT2

Nhóm: 2

I. NỘI DUNG ĐÃ THỰC HÀNH

Hệ thống Employee Management API là một RESTful Web Service được thiết kế theo kiến trúc 3 tầng chuẩn, sử dụng Spring Boot 3.x, kết nối cơ sở dữ liệu lưu trữ tạm thời H2 In-Memory Database thông qua Spring Data JPA. Hệ thống tích hợp chặt chẽ cơ chế kiểm tra dữ liệu đầu vào (Jakarta Validation), ghi log hệ thống (SLF4J) và lập trình hướng khía cạnh (Spring AOP).

1. Kiến trúc 3 tầng chuẩn (Standard Architecture)

- Tầng Controller (EmployeeController): Tiếp nhận trực tiếp các HTTP Requests (GET, POST, PUT, DELETE) từ Client/Postman. Chịu trách nhiệm bọc dữ liệu phản hồi trong ResponseEntity đi kèm HTTP Status Code chuẩn quốc tế (200, 201, 204, 400, 404). Tích hợp @Valid để kiểm tra tính hợp lệ của dữ liệu trước khi xử lý.
- Tầng Service (EmployeeService): Nơi thực thi toàn bộ logic nghiệp vụ của hệ thống. Nhận dữ liệu hợp lệ từ Controller, điều phối và kiểm tra sự tồn tại của nhân viên qua ID. Đưa ra các cảnh báo bằng Log hệ thống và ném ngoại lệ (RuntimeException) khi phát hiện vi phạm luồng nghiệp vụ.
- Tầng Repository (EmployeeRepository): Kế thừa JpaRepository từ Spring Data JPA, thiết lập giao tiếp tự động xuống cơ sở dữ liệu H2 mà không cần cấu hình SQL thủ công, giúp tối ưu hóa thời gian thực thi các tác vụ CRUD.

2. Luồng xử lý nghiệp vụ cốt lõi (Core Logic)

- Lấy danh sách nhân viên (GET /api/employees): Truy quét toàn bộ dữ liệu hiện có thông qua findAll(), trả về danh sách dạng JSON Array (HTTP Status 200 OK).

- Xem chi tiết nhân viên (GET /api/employees/{id}): Tìm kiếm nhân viên dựa trên Path Variable id. Nếu tìm thấy trả về đối tượng JSON (200 OK); ngược lại, hệ thống ghi log cảnh báo và trả về trạng thái không tìm thấy (404 Not Found).
 - Thêm mới nhân viên (POST /api/employees): Tiếp nhận Request Body có cấu trúc gồm fullName, department, salary. Hệ thống tự động kích hoạt bộ Validator để kiểm tra điều kiện dữ liệu và tự động sinh mã ID tăng dần thông qua cấu hình @GeneratedValue trong Entity, trả về thực thể vừa tạo (201 Created).
 - Cập nhật toàn bộ thông tin (PUT /api/employees/{id}): Kiểm tra sự tồn tại của ID. Nếu ID tồn tại, hệ thống tiến hành ghi đè dữ liệu mới (fullName, department, salary) lên bản ghi cũ và lưu lại (200 OK). Nếu không tìm thấy, hệ thống ném lỗi (404 Not Found).
 - Xóa nhân viên (DELETE /api/employees/{id}): Thực hiện kiểm tra sự tồn tại của ID nhân viên trước khi gọi hàm xóa dữ liệu. Xóa thành công hệ thống sẽ không trả về dữ liệu (204 No Content).
3. Ràng buộc dữ liệu & Quản lý phản hồi (Validation & Exception Handling)
- Jakarta Validation: * fullName và department: Bắt buộc điền, không được để trống hoặc null (@NotBlank).
 - salary: Mức lương nhân viên bắt buộc phải là số dương lớn hơn 0 (@Positive).
 - Global Exception Handling: Sử dụng @RestControllerAdvice kết hợp @ExceptionHandler để bắt trọn các ngoại lệ MethodArgumentNotValidException (khi vi phạm dữ liệu đầu vào) và trả về thông tin lỗi rõ ràng cho Client với mã lỗi 400 Bad Request.

II. CÔNG VIỆC CÁ NHÂN

Trong dự án Mini Project này, công việc cá nhân trọng tâm của em là Nghiên cứu, thiết kế và xây dựng lớp Aspect (LoggingAspect) ứng dụng lập trình hướng khía cạnh (Spring AOP) cùng các Annotation nhằm quản lý, giám sát hệ thống và ghi log tự động.

1. Thiết lập cấu hình và Xây dựng Lớp Aspect

- Khai báo và cấu hình phụ thuộc thư viện spring-boot-starter-aop trong file quản lý dự án.

- Xây dựng class `LoggingAspect` nằm trong package `aspect`, đánh dấu lớp này là một thành phần xử lý của Spring IoC thông qua hai Annotation cốt lõi:
 - `@Aspect`: Định nghĩa class đóng vai trò là một khía cạnh (Aspect) chuyên biệt để cắt ngang các luồng xử lý thông thường.
 - `@Component`: Đăng ký lớp vào Spring Container để tự động phát hiện và kích hoạt.

III. KẾT QUẢ ĐẠT ĐƯỢC

1. Kiến trúc & Tổ chức mã nguồn sạch: Dự án phân tách rõ ràng thành các tầng độc lập (Controller, Service, Repository, Entity, Aspect). Mã nguồn tuân thủ nghiêm ngặt chuẩn RESTful API và quy chuẩn đặt tên tài nguyên.
2. Tách biệt hoàn toàn mã nguồn kiểm soát (Separation of Concerns): Nhờ việc áp dụng Spring AOP do em triển khai, toàn bộ mã nguồn liên quan đến đo lường thời gian, ghi log lịch sử gọi hàm đã được tách rời 100% ra khỏi logic nghiệp vụ của dự án. Tầng Service và Controller hoàn toàn sạch đẹp, không bị chồng chéo mã nguồn Log dư thừa.
3. Tích hợp Cơ sở dữ liệu và Validation mượt mà: Hệ thống H2 Database tự động khởi tạo dữ liệu mẫu khi ứng dụng khởi chạy. Bộ lọc Validation chặn đứng toàn bộ các request gửi thiếu thông tin hoặc sai định mức lương từ phía Client một cách tự động.
4. Hệ thống Unit Test hoàn chỉnh: Đã viết đầy đủ và vượt qua toàn bộ 5 test case cho `EmployeeService` (sử dụng Mockito) và 4 test case cho `EmployeeController` (sử dụng MockMvc), đảm bảo ứng dụng không phát sinh lỗi tiềm ẩn khi vận hành.

- **Link dự án GitHub:**

V. KHÓ KHĂN VÀ VẤN ĐỀ GẶP PHẢI

- Khó khăn khi viết biểu thức Pointcut trong AOP: Thời gian đầu em còn lúng túng khi cấu hình chuỗi AspectJ Expression cho các Advice, dẫn đến việc viết sai cú pháp khiến Aspect không nhận diện hoặc nhận diện sai các Package cần cắt trúng (nhầm lẫn giữa package controller và service).
- Quản lý giá trị trả về trong `@Around` advice: Khi viết cấu trúc `@Around`, ban đầu em quên không cấu hình giá trị trả về (return object;) sau lệnh `proceed()`, dẫn đến việc các Controller đều bị trả về dữ liệu trống (null)

mặc dù Service vẫn chạy chính xác. Sau đó em đã tìm ra nguyên nhân và sửa lại bằng cách bọc đối tượng trả về chuẩn xác.

- Kiểm thử đa luồng và giám sát Log: Khi chạy đồng thời nhiều Test Case bằng MockMvc, các dòng Log in ra bảng điều khiển Console của IDE bị xáo trộn liên tục, gây khó khăn cho việc theo dõi thứ tự xuất hiện của @Before, @AfterReturning và @Around nhằm chứng minh tính đúng đắn của tiến trình.

V. KINH NGHIỆM RÚT RA

- Kinh nghiệm rút ra từ bài thực hành: Lập trình hướng khía cạnh (AOP) là một giải pháp cực kỳ mạnh mẽ để giải quyết các bài toán mang tính chất hệ thống (Cross-cutting Concerns) như Logging, Security, Transaction. Tuy nhiên, cần phải đặc biệt lưu ý kiểm soát luồng dữ liệu khi dùng @Around để tránh làm đứt gãy luồng trả về gốc của phương thức.
- Lưu ý cho các buổi sau: Cần đọc kỹ tài liệu hướng dẫn và thực hành nhuần nhuyễn cách viết Pointcut Expression để áp dụng chính xác cho từng môi trường doanh nghiệp. Chuẩn bị sẵn file cấu hình file log (logback-spring.xml) để quản lý đầu ra của SLF4J chuyên nghiệp hơn.

VII. KẾT LUẬN

- Khả năng hiểu bài của cá nhân được đánh giá: 8.5/10 (Tốt).
- Thông qua Mini Project này, em đã làm chủ được cách xây dựng hoàn chỉnh hệ thống RESTful API từ đầu, nắm vững kiến thức nâng cao về Spring Data JPA, kiểm soát dữ liệu chặt chẽ bằng Validation, biết cách viết Unit Test chuyên sâu và đặc biệt là áp dụng thành thạo Spring AOP kết hợp SLF4J để quản trị hệ thống một cách tự động và chuyên nghiệp theo chuẩn công nghiệp.